Replication Instructions

ISE Coursework — Bug Report Classification

Overview

These instructions explain exactly how to reproduce the results reported in the coursework. The experiment is fully deterministic — given the same data and the same code, you will get the same numbers every time. The only requirement is that you use the same random seeds, which are hardcoded into the script (train/test split seeds 0–29, Random Forest internal seed 42).

Step 1 — Clone the Repository

```
git clone https://github.com/NeellNRA/ise-bug-classification.git
cd ise-bug-classification
```

Step 2 — Set Up the Environment

```
pip install scikit-learn>=1.3 scipy>=1.11 numpy>=1.24 pandas>=2.0
```

Verify it worked:

```
python -c "import sklearn, scipy, numpy, pandas; print('All packages OK')"
```

Step 3 — Download the Dataset

The dataset is not included in the repository (to keep it lightweight). Download the five CSV files from the official lab repo:

```
https://github.com/ideas-labo/ISE/tree/main/lab1
```

Create a folder called `data/` inside the cloned repo and place all five CSVs there:

```
mkdir data
# move TensorFlow.csv, PyTorch.csv, Keras.csv, MXNet.csv, Caffe.csv into data/
```

Step 4 — Run the Experiment

```
python classifier_fixed.py --data_dir ./data --output_dir ./results
```

This will run 30 experiments per project for both the baseline and the proposed solution. It takes 3–8 minutes depending on your hardware.

Step 5 — Check the Results

Once finished, open `results/summary.csv`. The mean and standard deviation values in that file should match Table 1 in the report, and the p-values and A12 scores should match Table 2. Any differences will be in the last decimal place due to floating-point rounding in display.

The raw per-run files (e.g., `results/TensorFlow_solution_raw.csv`) contain all 30 individual scores and can be used to recompute any statistic independently.

Why the Results Are Deterministic

The train/test splits are generated with `random_state=seed` for seed values 0 to 29, so the same split is always produced for the same seed on the same data. The Random Forest uses `random_state=42` internally. Stratified splitting (`stratify=labels`) ensures the class ratio is preserved in every split. No resampling, data augmentation, or external randomness is involved.

Pre-Computed Results

The `results/` folder in the repository already contains the pre-computed CSVs from the original run. These can be used to verify the reported numbers without re-running the experiment.